

Implementation of Ridge,Lasso Regression

```
'''Cleaning the data'''
```

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
In [2]: dataset=pd.read_csv('/Users/mujeebmohammed/Downloads/Ridge Lasso Elastic
```

```
In [3]: dataset.head()
```

```
Out[3]:
```

	day	month	year	Temperature	RH	Ws	Rain	FFMC	DMC	DC	ISI	BUI	FW
0	01	06	2012	29	57	18	0	65.7	3.4	7.6	1.3	3.4	0
1	02	06	2012	29	61	13	1.3	64.4	4.1	7.6	1	3.9	0
2	03	06	2012	26	82	22	13.1	47.1	2.5	7.1	0.3	2.7	0
3	04	06	2012	25	89	13	2.5	28.6	1.3	6.9	0	1.7	0
4	05	06	2012	27	77	16	0	64.8	3	14.2	1.2	3.9	0

```
In [4]: dataset.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 246 entries, 0 to 245
Data columns (total 14 columns):
#   Column          Non-Null Count  Dtype
---  -
0   day              246 non-null    str
1   month            245 non-null    str
2   year             245 non-null    str
3   Temperature      245 non-null    str
4   RH               245 non-null    str
5   Ws               245 non-null    str
6   Rain             245 non-null    str
7   FFMC             245 non-null    str
8   DMC              245 non-null    str
9   DC               245 non-null    str
10  ISI              245 non-null    str
11  BUI              245 non-null    str
12  FWI              245 non-null    str
13  Classes          244 non-null    str
dtypes: str(14)
memory usage: 27.0 KB
```

```
'''data cleaning'''
```

```
In [5]: dataset.isnull().sum()
```

```
Out[5]: day          0
        month       1
        year        1
        Temperature  1
         RH         1
         Ws         1
        Rain        1
        FFMC        1
        DMC         1
        DC          1
        ISI         1
        BUI         1
        FWI         1
        Classes     2
        dtype: int64
```

```
In [6]: dataset[dataset.isnull().any(axis=1)]
```

```
Out[6]:
```

	day	month	year	Temperature	RH	Ws	Rain	FFMC	DMC	DC	ISI
122	Sidi-Bel Abbes Region Dataset	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
167	14	07	2012	37	37	18	0.2	88.9	12.9	14.6 9	12.5

```
In [7]: dataset = dataset.reset_index(drop=True)
        dataset.loc[:121, 'Region'] = 0
        dataset.loc[122:, 'Region'] = 1
```

```
In [8]: df=dataset
```

```
In [9]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 246 entries, 0 to 245
Data columns (total 15 columns):
#   Column          Non-Null Count  Dtype
---  -
0   day              246 non-null    str
1   month           245 non-null    str
2   year            245 non-null    str
3   Temperature     245 non-null    str
4   RH              245 non-null    str
5   Ws              245 non-null    str
6   Rain            245 non-null    str
7   FFMC            245 non-null    str
8   DMC             245 non-null    str
9   DC              245 non-null    str
10  ISI             245 non-null    str
11  BUI             245 non-null    str
12  FWI             245 non-null    str
13  Classes         244 non-null    str
14  Region          246 non-null    float64
dtypes: float64(1), str(14)
memory usage: 29.0 KB
```

```
In [10]: df[['Region']] = df[['Region']].astype(int)
```

```
In [11]: df.head()
```

```
Out[11]:
```

	day	month	year	Temperature	RH	Ws	Rain	FFMC	DMC	DC	ISI	BUI	FWI
0	01	06	2012	29	57	18	0	65.7	3.4	7.6	1.3	3.4	0
1	02	06	2012	29	61	13	1.3	64.4	4.1	7.6	1	3.9	0
2	03	06	2012	26	82	22	13.1	47.1	2.5	7.1	0.3	2.7	0
3	04	06	2012	25	89	13	2.5	28.6	1.3	6.9	0	1.7	0
4	05	06	2012	27	77	16	0	64.8	3	14.2	1.2	3.9	0

```
In [12]: df.isnull().sum()
```

```
Out[12]: day          0
month        1
year         1
Temperature  1
RH           1
Ws           1
Rain         1
FFMC         1
DMC          1
DC           1
ISI          1
BUI          1
FWI          1
Classes      2
Region       0
dtype: int64
```

```
In [13]: #removing null values
df=df.dropna().reset_index(drop=True)
```

```
In [14]: df.head()
```

```
Out[14]:
```

	day	month	year	Temperature	RH	Ws	Rain	FFMC	DMC	DC	ISI	BUI	FWI
0	01	06	2012	29	57	18	0	65.7	3.4	7.6	1.3	3.4	0
1	02	06	2012	29	61	13	1.3	64.4	4.1	7.6	1	3.9	0
2	03	06	2012	26	82	22	13.1	47.1	2.5	7.1	0.3	2.7	0
3	04	06	2012	25	89	13	2.5	28.6	1.3	6.9	0	1.7	0
4	05	06	2012	27	77	16	0	64.8	3	14.2	1.2	3.9	0

```
In [15]: df.isnull().sum()
```

```
Out[15]: day          0
         month        0
         year         0
         Temperature  0
         RH           0
         Ws           0
         Rain         0
         FFMC         0
         DMC          0
         DC           0
         ISI          0
         BUI          0
         FWI          0
         Classes      0
         Region       0
         dtype: int64
```

```
In [16]: df.iloc[[122]]
```

```
Out[16]:
```

	day	month	year	Temperature	RH	Ws	Rain	FFMC	DMC	DC	ISI	BUI	F
122	day	month	year	Temperature	RH	Ws	Rain	FFMC	DMC	DC	ISI	BUI	F

```
In [17]: #remove the 122nd row
df=df.drop(122).reset_index(drop=True)
df.iloc[[122]]
df.columns
```

```
Out[17]: Index(['day', 'month', 'year', 'Temperature', ' RH', ' Ws', 'Rain ', 'FF
MC',
               'DMC', 'DC', 'ISI', 'BUI', 'FWI', 'Classes ', 'Region'],
              dtype='str')
```

```
In [18]: #fix spaces in columns names
df.columns=df.columns.str.strip()
df.columns
```

```
Out[18]: Index(['day', 'month', 'year', 'Temperature', 'RH', 'Ws', 'Rain', 'FFM
C',
               'DMC', 'DC', 'ISI', 'BUI', 'FWI', 'Classes', 'Region'],
              dtype='str')
```

```
In [19]: df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 243 entries, 0 to 242
Data columns (total 15 columns):
#   Column          Non-Null Count  Dtype
---  -
0   day              243 non-null    str
1   month            243 non-null    str
2   year             243 non-null    str
3   Temperature      243 non-null    str
4   RH               243 non-null    str
5   Ws               243 non-null    str
6   Rain             243 non-null    str
7   FFMC             243 non-null    str
8   DMC              243 non-null    str
9   DC               243 non-null    str
10  ISI              243 non-null    str
11  BUI              243 non-null    str
12  FWI              243 non-null    str
13  Classes          243 non-null    str
14  Region           243 non-null    int64
dtypes: int64(1), str(14)
memory usage: 28.6 KB

```

```

In [20]: ##changes the required columns as integer data
df.columns

```

```

Out[20]: Index(['day', 'month', 'year', 'Temperature', 'RH', 'Ws', 'Rain', 'FFM
C',
               'DMC', 'DC', 'ISI', 'BUI', 'FWI', 'Classes', 'Region'],
              dtype='str')

```

```

In [21]: df.columns = df.columns.str.strip()
df[['month','day','year','Temperature','RH','Ws']].astype(int)

```

```

Out[21]:

```

	month	day	year	Temperature	RH	Ws
0	6	1	2012	29	57	18
1	6	2	2012	29	61	13
2	6	3	2012	26	82	22
3	6	4	2012	25	89	13
4	6	5	2012	27	77	16
...	...	...	...	...	...	...
238	9	26	2012	30	65	14
239	9	27	2012	28	87	15
240	9	28	2012	27	87	29
241	9	29	2012	24	54	18
242	9	30	2012	24	64	15

243 rows x 6 columns

```
In [22]: #changing the other columns to float datatype
objects=[features for features in df.columns if df[features].dtype=='o']
```

```
In [23]: for i in objects:
          if i!='classes':
              df[i]=df[i].astype(float)
```

```
In [24]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 243 entries, 0 to 242
Data columns (total 15 columns):
#   Column          Non-Null Count  Dtype
---  -
0   day              243 non-null   str
1   month            243 non-null   str
2   year             243 non-null   str
3   Temperature      243 non-null   str
4   RH               243 non-null   str
5   Ws               243 non-null   str
6   Rain             243 non-null   str
7   FPMC             243 non-null   str
8   DMC              243 non-null   str
9   DC               243 non-null   str
10  ISI              243 non-null   str
11  BUI              243 non-null   str
12  FWI              243 non-null   str
13  Classes          243 non-null   str
14  Region           243 non-null   int64
dtypes: int64(1), str(14)
memory usage: 28.6 KB
```

```
In [25]: df.describe()
```

```
Out[25]:
```

	Region
count	243.000000
mean	0.497942
std	0.501028
min	0.000000
25%	0.000000
50%	0.000000
75%	1.000000
max	1.000000

```
In [26]: df.head()
```

```
Out[26]:
```

	day	month	year	Temperature	RH	Ws	Rain	FFMC	DMC	DC	ISI	BUI	FWI
0	01	06	2012	29	57	18	0	65.7	3.4	7.6	1.3	3.4	0
1	02	06	2012	29	61	13	1.3	64.4	4.1	7.6	1	3.9	0
2	03	06	2012	26	82	22	13.1	47.1	2.5	7.1	0.3	2.7	0
3	04	06	2012	25	89	13	2.5	28.6	1.3	6.9	0	1.7	0
4	05	06	2012	27	77	16	0	64.8	3	14.2	1.2	3.9	0

""EDA and feature engineering""

```
In [27]: #lets save the cleaned dataset
df.to_csv('Algerian_forest_fires_cleaned.csv',index=False)
```

```
In [28]: df=pd.read_csv('/Users/mujeebmohammed/Documents/DS/machine learning/Algerian_forest_fires_cleaned.csv')
```

```
In [29]: df.head()
```

```
Out[29]:
```

	day	month	year	Temperature	RH	Ws	Rain	FFMC	DMC	DC	ISI	BUI	FWI
0	1	6	2012	29	57	18	0.0	65.7	3.4	7.6	1.3	3.4	0
1	2	6	2012	29	61	13	1.3	64.4	4.1	7.6	1.0	3.9	0
2	3	6	2012	26	82	22	13.1	47.1	2.5	7.1	0.3	2.7	0
3	4	6	2012	25	89	13	2.5	28.6	1.3	6.9	0.0	1.7	0
4	5	6	2012	27	77	16	0.0	64.8	3.0	14.2	1.2	3.9	0

```
In [30]: #exploratory data analysis
#drop day,month,year
df_copy=df.drop(['day','month','year'],axis=1)
```

```
In [31]: ##encoding of the categories in classes
df_copy['Classes']=np.where(df_copy['Classes'].str.contains('not fire'),0,1)
```

```
In [32]: df_copy.head()
```

```
Out[32]:
```

	Temperature	RH	Ws	Rain	FFMC	DMC	DC	ISI	BUI	FWI	Classes	Region
0	29	57	18	0.0	65.7	3.4	7.6	1.3	3.4	0.5	0	0
1	29	61	13	1.3	64.4	4.1	7.6	1.0	3.9	0.4	0	0
2	26	82	22	13.1	47.1	2.5	7.1	0.3	2.7	0.1	0	0
3	25	89	13	2.5	28.6	1.3	6.9	0.0	1.7	0.0	0	0
4	27	77	16	0.0	64.8	3.0	14.2	1.2	3.9	0.5	0	0

```
In [33]: df_copy.tail()
```

Out [33]:

	Temperature	RH	Ws	Rain	FFMC	DMC	DC	ISI	BUI	FWI	Classes	Reg
238	30	65	14	0.0	85.4	16.0	44.5	4.5	16.9	6.5	1	
239	28	87	15	4.4	41.1	6.5	8.0	0.1	6.2	0.0	0	
240	27	87	29	0.5	45.9	3.5	7.9	0.4	3.4	0.2	0	
241	24	54	18	0.1	79.7	4.3	15.2	1.7	5.1	0.7	0	
242	24	64	15	0.2	67.3	3.8	16.5	1.2	4.8	0.5	0	

In [34]: *#categories in classes*
df_copy['Classes'].value_counts()

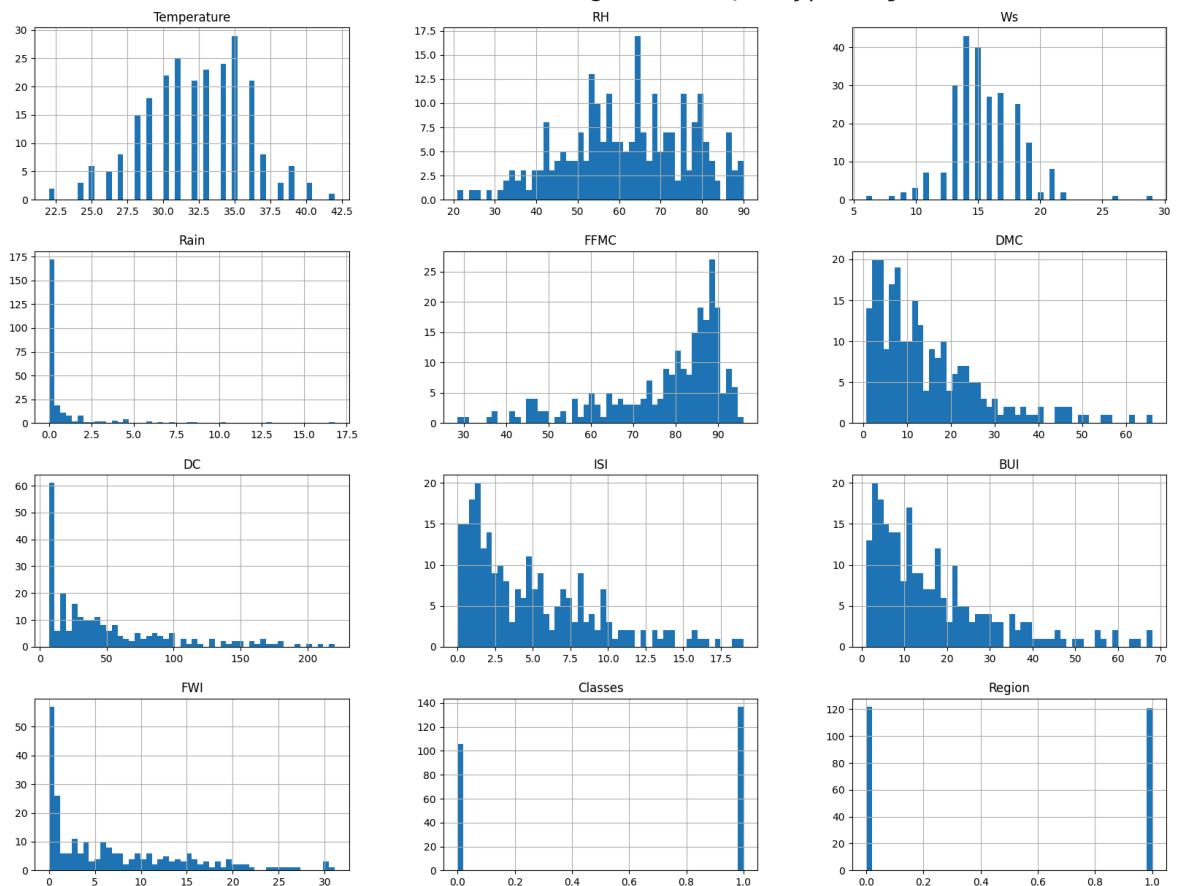
Out [34]:

```
Classes
1    137
0    106
Name: count, dtype: int64
```

In [35]: *#plot density plot for all features*
df_copy.hist(bins=50, figsize=(20, 15))

Out [35]:

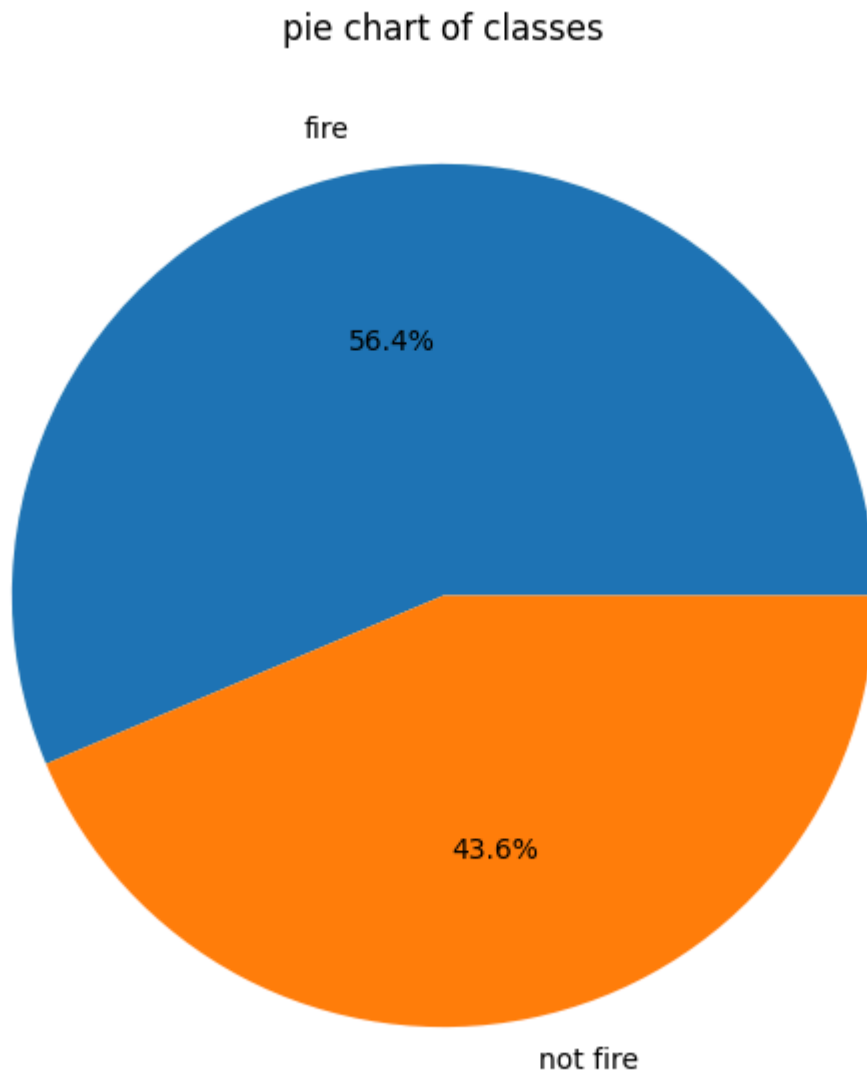
```
array([[<Axes: title={'center': 'Temperature'}>,
        <Axes: title={'center': 'RH'}>, <Axes: title={'center': 'Ws'}>],
       [<Axes: title={'center': 'Rain'}>,
        <Axes: title={'center': 'FFMC'}>,
        <Axes: title={'center': 'DMC'}>],
       [<Axes: title={'center': 'DC'}>, <Axes: title={'center': 'ISI'}>,
        <Axes: title={'center': 'BUI'}>],
       [<Axes: title={'center': 'FWI'}>,
        <Axes: title={'center': 'Classes'}>,
        <Axes: title={'center': 'Region'}>]], dtype=object)
```



```
In [36]: #percentage for pie chart
percentage=df_copy['Classes'].value_counts(normalize=True)*100
```

```
In [37]: #plotting piechart

classlabels=['fire','not fire']
plt.figure(figsize=(12,7))
plt.pie(percentage, labels=classlabels,autopct='%1.1f%%')
plt.title('pie chart of classes')
plt.show()
```



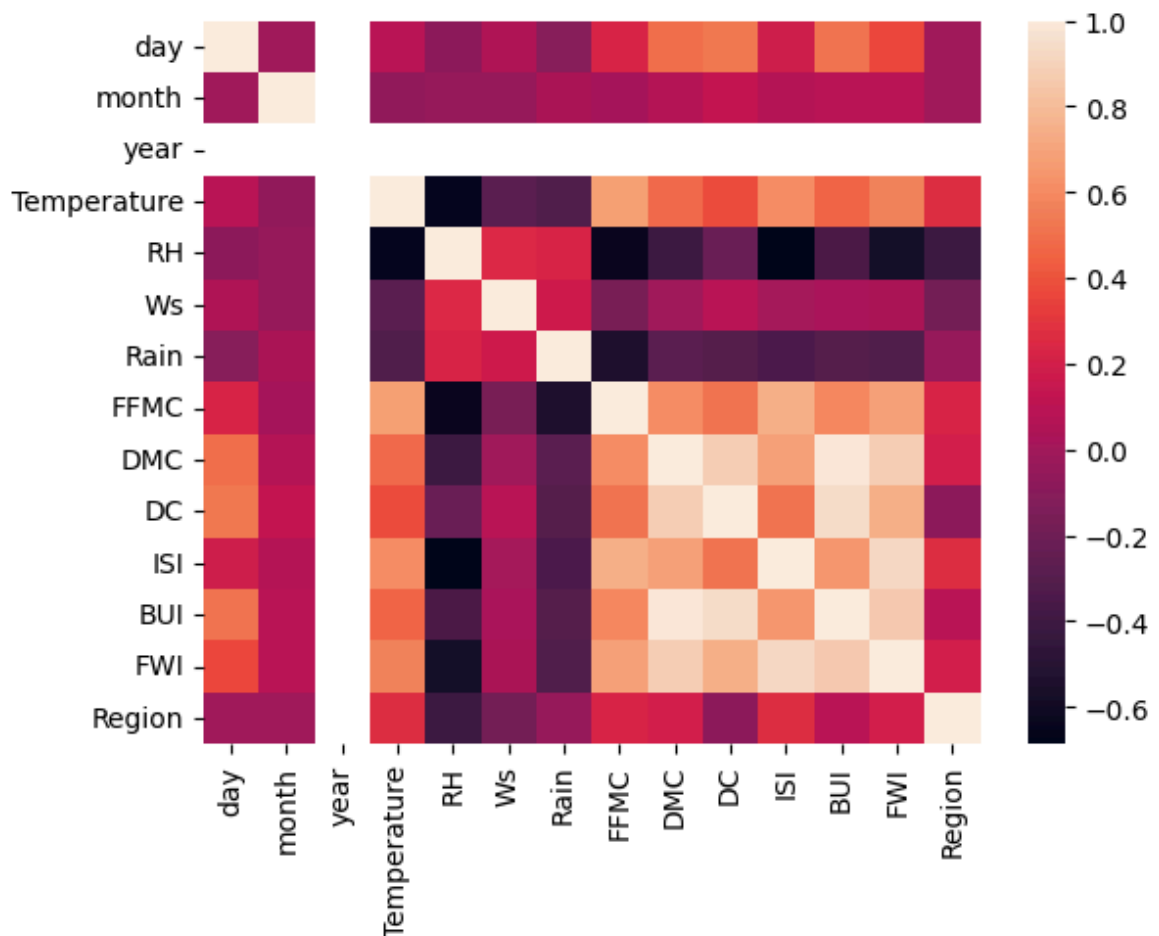
```
In [38]: #correlation
df_copy.corr()
```

Out [38]:

	Temperature	RH	Ws	Rain	FFMC	DMC
Temperature	1.000000	-0.651400	-0.284510	-0.326492	0.676568	0.485687
RH	-0.651400	1.000000	0.244048	0.222356	-0.644873	-0.408519
Ws	-0.284510	0.244048	1.000000	0.171506	-0.166548	-0.000721
Rain	-0.326492	0.222356	0.171506	1.000000	-0.543906	-0.288773
FFMC	0.676568	-0.644873	-0.166548	-0.543906	1.000000	0.603608
DMC	0.485687	-0.408519	-0.000721	-0.288773	0.603608	1.000000
DC	0.376284	-0.226941	0.079135	-0.298023	0.507397	0.875925
ISI	0.603871	-0.686667	0.008532	-0.347484	0.740007	0.680454
BUI	0.459789	-0.353841	0.031438	-0.299852	0.592011	0.982248
FWI	0.566670	-0.580957	0.032368	-0.324422	0.691132	0.875864
Classes	0.516015	-0.432161	-0.069964	-0.379097	0.769492	0.585658
Region	0.269555	-0.402682	-0.181160	-0.040013	0.222241	0.192089

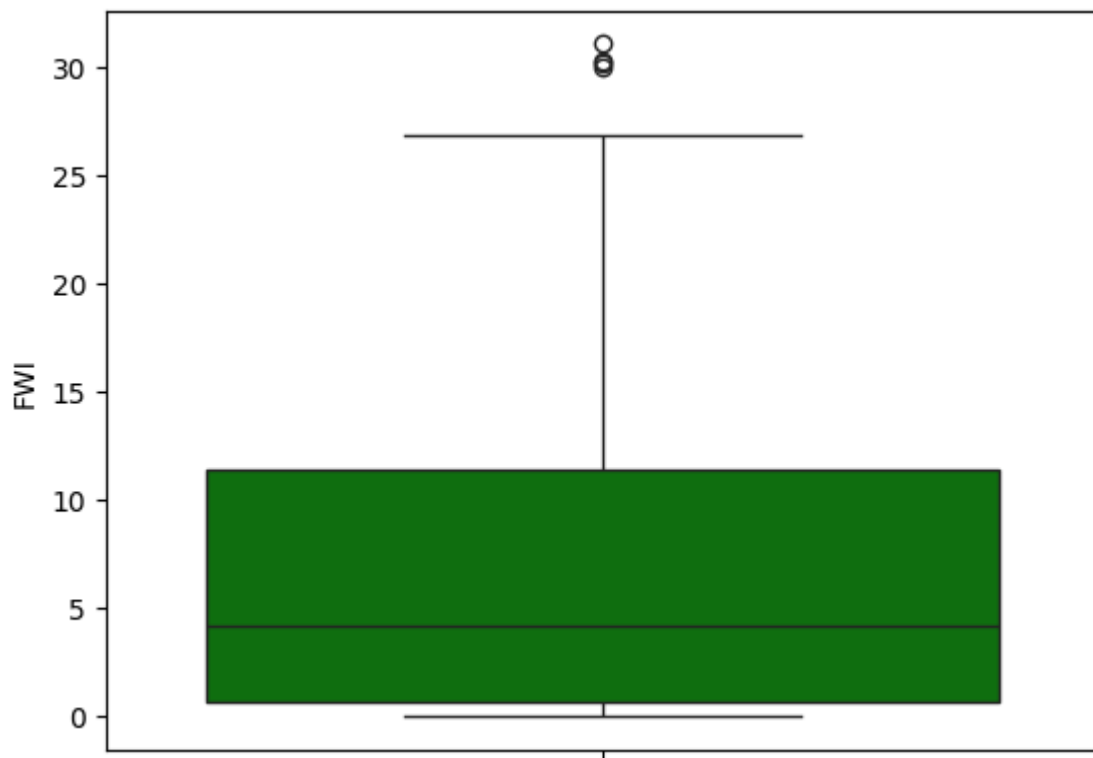
In [39]: `sns.heatmap(df.corr(numeric_only=True))`

Out [39]: <Axes: >



In [40]: `##box plots`
`sns.boxplot(df['FWI'],color='g')`

Out[40]: <Axes: ylabel='FWI'>



```
In [41]: df.head()
```

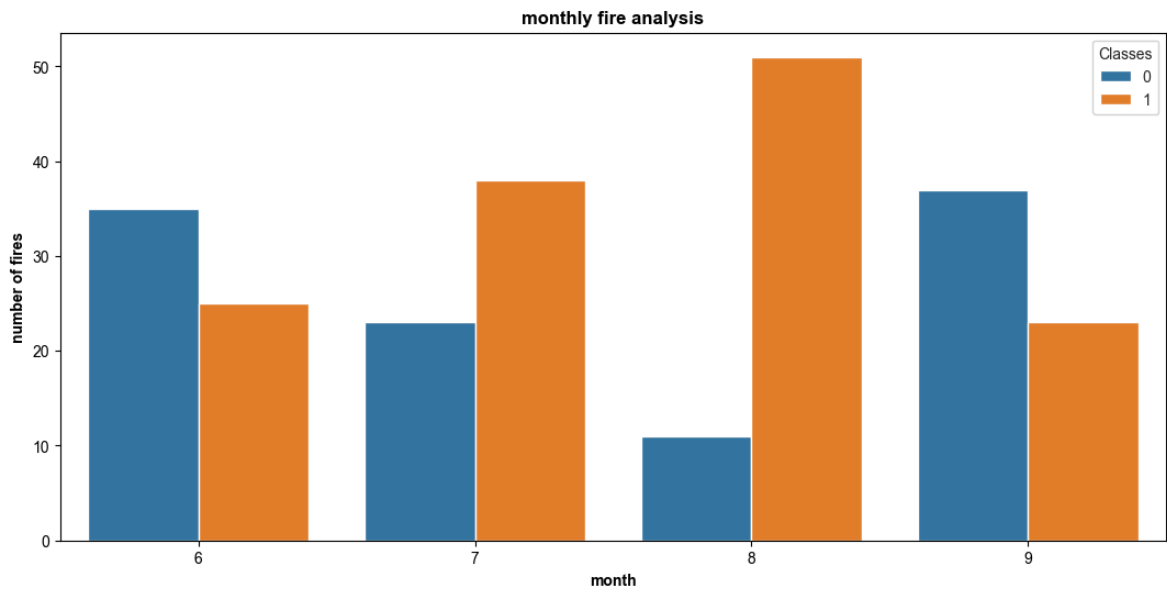
```
Out[41]:
```

	day	month	year	Temperature	RH	Ws	Rain	FFMC	DMC	DC	ISI	BUI	FWI
0	1	6	2012	29	57	18	0.0	65.7	3.4	7.6	1.3	3.4	0
1	2	6	2012	29	61	13	1.3	64.4	4.1	7.6	1.0	3.9	0
2	3	6	2012	26	82	22	13.1	47.1	2.5	7.1	0.3	2.7	0
3	4	6	2012	25	89	13	2.5	28.6	1.3	6.9	0.0	1.7	0
4	5	6	2012	27	77	16	0.0	64.8	3.0	14.2	1.2	3.9	0

```
In [42]: df['Classes'] = np.where(df['Classes'].str.contains('not fire','fire'), 0, 1)
```

```
In [43]: #monthly fire analysis
dftemp=df.loc[df['Region']==1]
plt.subplots(figsize=(13,6))
sns.set_style('whitegrid')
sns.countplot(x='month',hue='Classes',data=df)
plt.ylabel('number of fires',weight='bold')
plt.xlabel('month',weight='bold')
plt.title('monthly fire analysis',weight='bold')
```

```
Out[43]: Text(0.5, 1.0, 'monthly fire analysis')
```



'''feature selection'''

```
In [44]: #drop month,day,year
df.drop(['day','month','year'], axis=1,inplace=True)
```

```
In [45]: df.head()
```

```
Out[45]:
```

	Temperature	RH	Ws	Rain	FFMC	DMC	DC	ISI	BUI	FWI	Classes	Region
0	29	57	18	0.0	65.7	3.4	7.6	1.3	3.4	0.5	0	0
1	29	61	13	1.3	64.4	4.1	7.6	1.0	3.9	0.4	0	0
2	26	82	22	13.1	47.1	2.5	7.1	0.3	2.7	0.1	0	0
3	25	89	13	2.5	28.6	1.3	6.9	0.0	1.7	0.0	0	0
4	27	77	16	0.0	64.8	3.0	14.2	1.2	3.9	0.5	0	0

```
In [46]: df['Classes'].value_counts()
```

```
Out[46]: Classes
1      137
0      106
Name: count, dtype: int64
```

```
In [47]: #encoding
df['Classes'] = df['Classes'].astype(str)
df['Classes'] = np.where(df['Classes'].str.contains('not fire'), 0, 1)
```

```
In [48]: df.head()
```

```
Out [48]:
```

	Temperature	RH	Ws	Rain	FFMC	DMC	DC	ISI	BUI	FWI	Classes	Region
0	29	57	18	0.0	65.7	3.4	7.6	1.3	3.4	0.5	1	0
1	29	61	13	1.3	64.4	4.1	7.6	1.0	3.9	0.4	1	0
2	26	82	22	13.1	47.1	2.5	7.1	0.3	2.7	0.1	1	0
3	25	89	13	2.5	28.6	1.3	6.9	0.0	1.7	0.0	1	0
4	27	77	16	0.0	64.8	3.0	14.2	1.2	3.9	0.5	1	0

```
In [49]: df.tail()
```

```
Out [49]:
```

	Temperature	RH	Ws	Rain	FFMC	DMC	DC	ISI	BUI	FWI	Classes	Reg
238	30	65	14	0.0	85.4	16.0	44.5	4.5	16.9	6.5	1	
239	28	87	15	4.4	41.1	6.5	8.0	0.1	6.2	0.0	1	
240	27	87	29	0.5	45.9	3.5	7.9	0.4	3.4	0.2	1	
241	24	54	18	0.1	79.7	4.3	15.2	1.7	5.1	0.7	1	
242	24	64	15	0.2	67.3	3.8	16.5	1.2	4.8	0.5	1	

```
In [50]: df['Classes'].value_counts()
```

```
Out [50]: Classes
1      243
Name: count, dtype: int64
```

```
In [51]: #independent and dependent features
X=df.drop('FWI',axis=1)
y=df['FWI']
```

```
In [52]: X.head()
```

```
Out [52]:
```

	Temperature	RH	Ws	Rain	FFMC	DMC	DC	ISI	BUI	Classes	Region
0	29	57	18	0.0	65.7	3.4	7.6	1.3	3.4	1	0
1	29	61	13	1.3	64.4	4.1	7.6	1.0	3.9	1	0
2	26	82	22	13.1	47.1	2.5	7.1	0.3	2.7	1	0
3	25	89	13	2.5	28.6	1.3	6.9	0.0	1.7	1	0
4	27	77	16	0.0	64.8	3.0	14.2	1.2	3.9	1	0

```
In [53]: y
```

```
Out [53]: 0      0.5
          1      0.4
          2      0.1
          3      0.0
          4      0.5
          ...
          238    6.5
          239    0.0
          240    0.2
          241    0.7
          242    0.5
          Name: FWI, Length: 243, dtype: float64
```

```
In [54]: #train test split
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.25,random_
```

```
In [55]: X_train.shape,X_test.shape
```

```
Out [55]: ((182, 11), (61, 11))
```

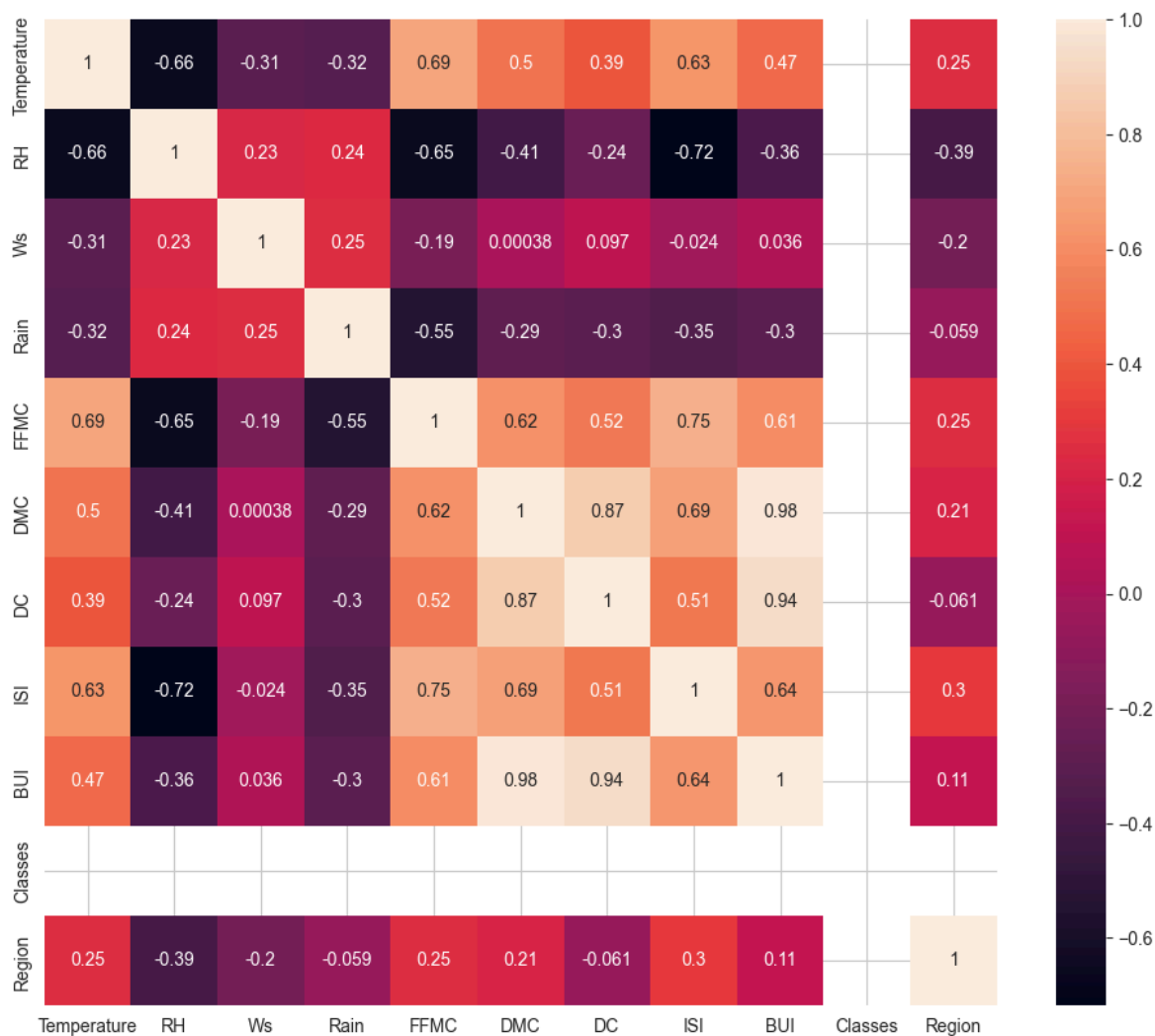
```
In [56]: #feature selection based on correlation
X_train.corr()
```

```
Out [56]:
```

	Temperature	RH	Ws	Rain	FFMC	DMC
Temperature	1.000000	-0.656095	-0.305977	-0.317512	0.694768	0.498173
RH	-0.656095	1.000000	0.225736	0.241656	-0.653023	-0.414601
Ws	-0.305977	0.225736	1.000000	0.251932	-0.190076	0.000379
Rain	-0.317512	0.241656	0.251932	1.000000	-0.545491	-0.289754
FFMC	0.694768	-0.653023	-0.190076	-0.545491	1.000000	0.620807
DMC	0.498173	-0.414601	0.000379	-0.289754	0.620807	1.000000
DC	0.390684	-0.236078	0.096576	-0.302341	0.524101	0.868647
ISI	0.629848	-0.717804	-0.023558	-0.345707	0.750799	0.685656
BUI	0.473609	-0.362317	0.035633	-0.300964	0.607210	0.983175
Classes	NaN	NaN	NaN	NaN	NaN	NaN
Region	0.254549	-0.394665	-0.199969	-0.059022	0.249514	0.212582

```
In [57]: #check for multicollinearity
plt.figure(figsize=(12,10))
corr=X_train.corr()
sns.heatmap(corr,annot=True)
```

```
Out [57]: <Axes: >
```



```
In [58]: def correlation(dataset,threshold):
col_corr=set()
corr_matrix=dataset.corr()
for i in range(len(corr_matrix.columns)):
    for j in range(i):
        if abs(corr_matrix.iloc[i,j])>threshold:
            colname=corr_matrix.columns[i]
            col_corr.add(colname)
return col_corr
```

```
In [59]: #threshold--Domain expertise
corr_features=correlation(X_train,0.85)
```

```
In [60]: #drop features when correlations is more than 0.85
X_train.drop(corr_features,axis=1,inplace=True)
X_test.drop(corr_features,axis=1,inplace=True)
X_train.shape,X_test.shape
```

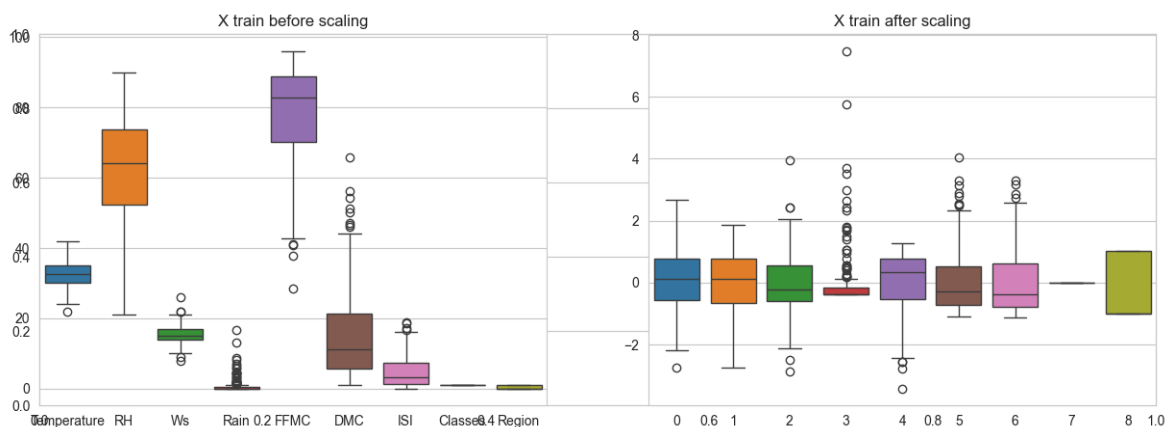
```
Out[60]: ((182, 9), (61, 9))
```

```
In [61]: #feature scaling or standardization
from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
X_train_scaled=scaler.fit_transform(X_train)
X_test_scaled=scaler.transform(X_test)
X_train_scaled
```

```
Out[61]: array([[ -0.84284248,  0.78307967,  1.29972026, ..., -0.62963326,
         0.          , -0.98907071],
        [ -0.30175842,  0.64950844, -0.59874754, ..., -0.93058524,
         0.          ,  1.01105006],
        [  2.13311985, -2.08870172, -0.21905398, ...,  2.7271388 ,
         0.          ,  1.01105006],
        ...,
        [ -1.9250106 ,  0.9166509 ,  0.54033314, ..., -1.06948615,
         0.          , -0.98907071],
        [  0.50986767, -0.21870454,  0.16063958, ...,  0.5973248 ,
         0.          ,  1.01105006],
        [ -0.57230045,  0.98343651,  2.05910739, ..., -0.86113478,
         0.          , -0.98907071]], shape=(182, 9))
```

```
In [62]: ##box plots to understand effect of standard scaler
plt.subplots(figsize=(15,5))
plt.subplot(1,2,1)
sns.boxplot(data=X_train)
plt.title('X train before scaling')
plt.subplot(1,2,2)
sns.boxplot(data=X_train_scaled)
plt.title('X train after scaling')
```

```
Out[62]: Text(0.5, 1.0, 'X train after scaling')
```

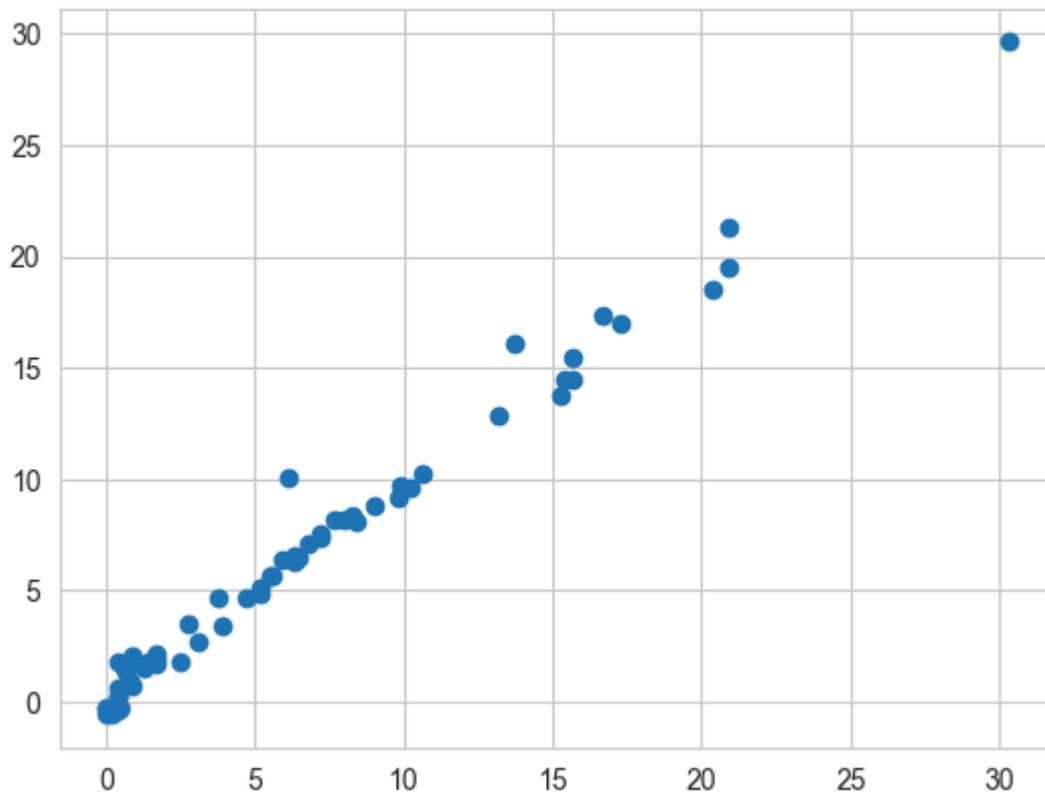


'''Model Training'''

```
In [64]: #linear regression model
from sklearn.metrics import mean_absolute_error, r2_score
from sklearn.linear_model import LinearRegression
linreg=LinearRegression()
linreg.fit(X_train_scaled,y_train)
y_pred=linreg.predict(X_test_scaled)
mae=mean_absolute_error(y_test,y_pred)
score=r2_score(y_test,y_pred)
print('MAE:',mae)
print('R2 score:',score)
plt.scatter(y_test,y_pred)
```

```
MAE: 0.5806361642310051
R2 score: 0.9830321384727897
```

```
Out[64]: <matplotlib.collections.PathCollection at 0x124acb390>
```

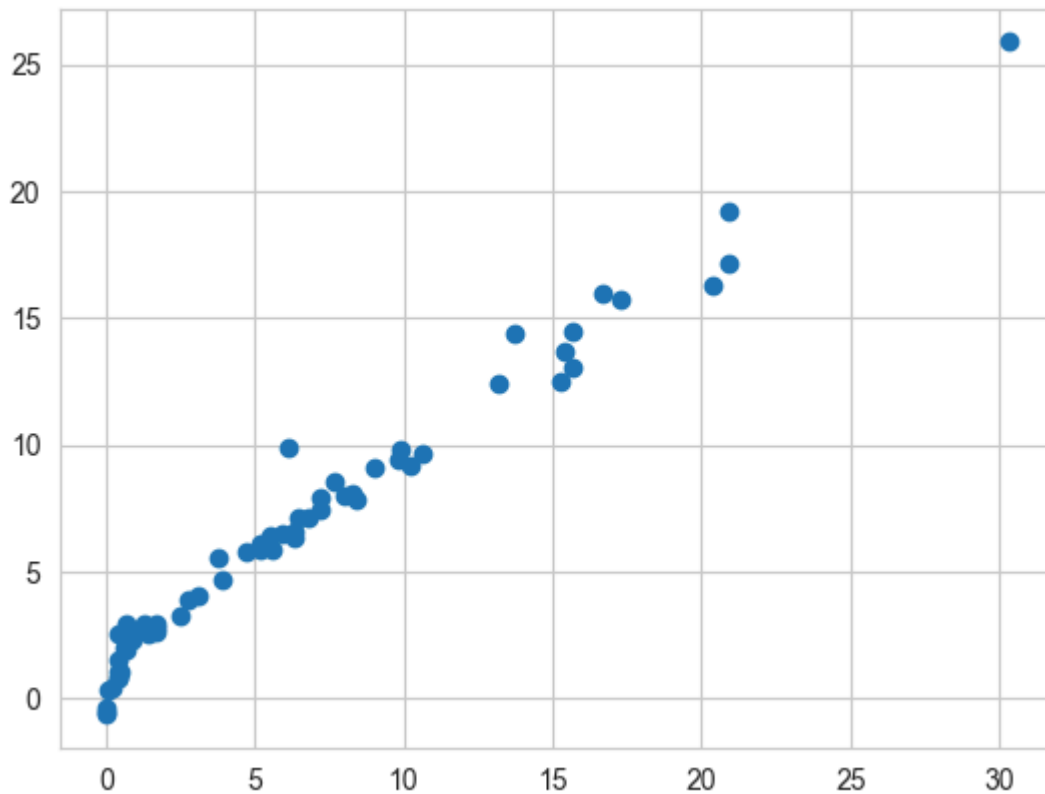


```
In [65]: #lasso regression
from sklearn.linear_model import Lasso
from sklearn.metrics import mean_absolute_error, r2_score
lasso=Lasso()
lasso.fit(X_train_scaled,y_train)
y_pred=lasso.predict(X_test_scaled)
mae=mean_absolute_error(y_test,y_pred)
score=r2_score(y_test,y_pred)
print('MAE:',mae)
print('R2 score:',score)
plt.scatter(y_test,y_pred)
```

MAE: 1.1331759949144085

R2 score: 0.9492020263112388

Out[65]: <matplotlib.collections.PathCollection at 0x12492a690>

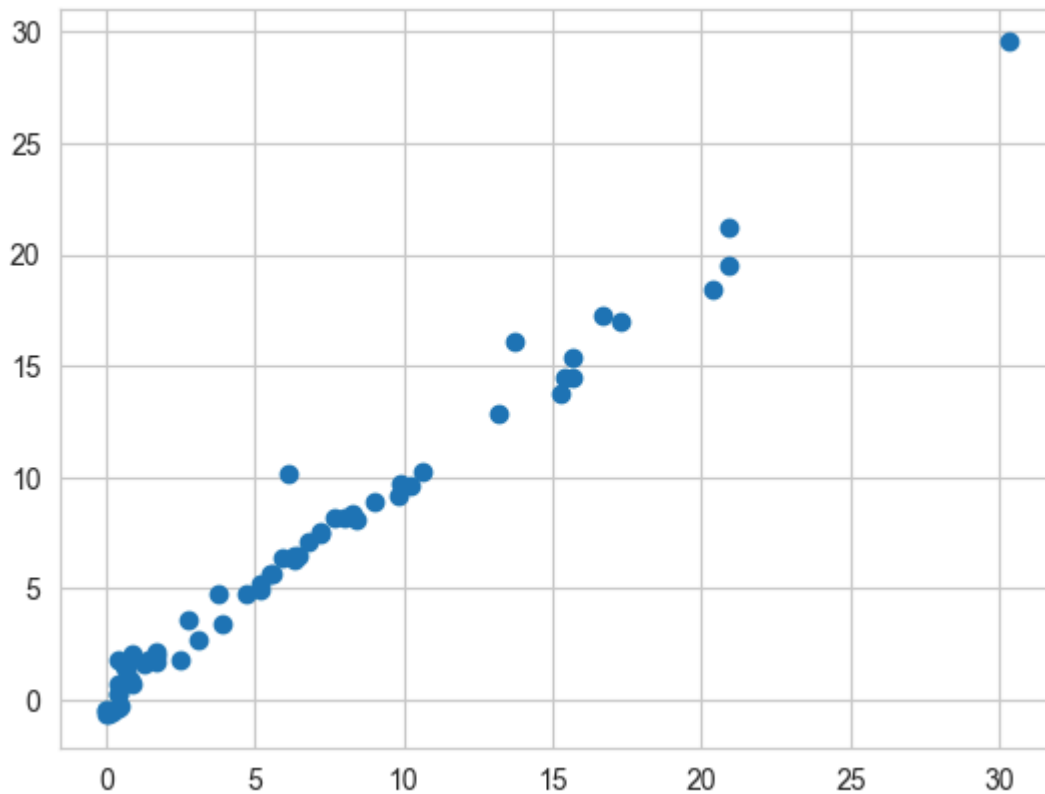


```
In [66]: #ridge regression
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, r2_score
ridge=Ridge()
ridge.fit(X_train_scaled,y_train)
y_pred=ridge.predict(X_test_scaled)
mae=mean_absolute_error(y_test,y_pred)
score=r2_score(y_test,y_pred)
print('MAE:',mae)
print('R2 score:',score)
plt.scatter(y_test,y_pred)
```

MAE: 0.5969613029130073

R2 score: 0.9824907652592586

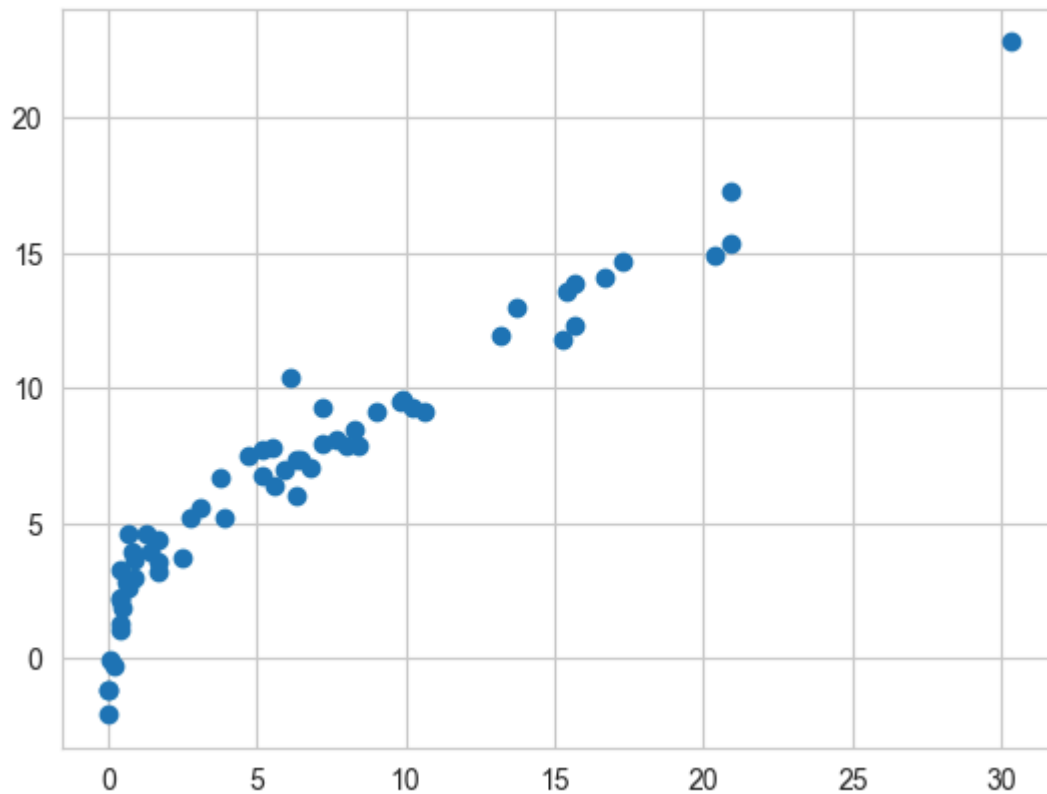
Out[66]: <matplotlib.collections.PathCollection at 0x1271376d0>



```
In [67]: #elastic net
from sklearn.linear_model import ElasticNet
from sklearn.metrics import mean_absolute_error, r2_score
elastic=ElasticNet()
elastic.fit(X_train_scaled,y_train)
y_pred=elastic.predict(X_test_scaled)
mae=mean_absolute_error(y_test,y_pred)
score=r2_score(y_test,y_pred)
print('MAE:',mae)
print('R2 score:',score)
plt.scatter(y_test,y_pred)
```

MAE: 1.928441451179673
R2 score: 0.868664603256521

Out[67]: <matplotlib.collections.PathCollection at 0x12720f6d0>



"""Hyperparameter tuning"""

```
In [68]: #cross validation lasso
from sklearn.linear_model import LassoCV
lasso_cv=LassoCV(cv=5)
lasso_cv.fit(X_train_scaled,y_train)
```

```
Out[68]: ▼ LassoCV ⓘ ?
          ► Parameters
```

```
In [69]: lasso_cv.alpha_
```

```
Out[69]: np.float64(0.0433104856814398)
```

```
In [70]: lasso_cv.alphas_
```

```
Out[70]: array([7.05853002, 6.58280872, 6.13914944, 5.72539132, 5.33951911,
4.97965339, 4.64404142, 4.33104857, 4.03915039, 3.76692517,
3.51304702, 3.27627941, 3.05546914, 2.84954075, 2.65749124,
2.47838523, 2.31135036, 2.15557308, 2.01029467, 1.87480753,
1.74845178, 1.63061198, 1.52071419, 1.41822315, 1.32263965,
1.23349817, 1.15036452, 1.0728338 , 1.00052839, 0.93309613,
0.87020857, 0.81155943, 0.75686304, 0.705853 , 0.65828087,
0.61391494, 0.57253913, 0.53395191, 0.49796534, 0.46440414,
0.43310486, 0.40391504, 0.37669252, 0.3513047 , 0.32762794,
0.30554691, 0.28495408, 0.26574912, 0.24783852, 0.23113504,
0.21555731, 0.20102947, 0.18748075, 0.17484518, 0.1630612 ,
0.15207142, 0.14182231, 0.13226397, 0.12334982, 0.11503645,
0.10728338, 0.10005284, 0.09330961, 0.08702086, 0.08115594,
0.0756863 , 0.0705853 , 0.06582809, 0.06139149, 0.05725391,
0.05339519, 0.04979653, 0.04644041, 0.04331049, 0.0403915 ,
0.03766925, 0.03513047, 0.03276279, 0.03055469, 0.02849541,
0.02657491, 0.02478385, 0.0231135 , 0.02155573, 0.02010295,
0.01874808, 0.01748452, 0.01630612, 0.01520714, 0.01418223,
0.0132264 , 0.01233498, 0.01150365, 0.01072834, 0.01000528,
0.00933096, 0.00870209, 0.00811559, 0.00756863, 0.00705853])
```

```
In [71]: lasso_cv.mse_path_
```

```
Out[71]: array([[54.04680834, 65.39074075, 54.89628985, 77.90299066, 39.3823684
6],
[46.99139504, 59.81506814, 49.29840831, 75.6645852 , 35.3175835
5],
[41.14908917, 53.9178721 , 42.49594889, 68.35708298, 31.7699612
3],
[36.05788169, 48.41773965, 36.66879347, 61.92819453, 28.6729681
6],
[31.62067285, 42.76466229, 31.68391904, 56.26841266, 24.9318329
5],
[27.75285367, 37.81922812, 27.42599138, 51.28216115, 21.5688864
5],
[24.3807738 , 33.49107651, 23.7950306 , 45.25481907, 18.6609416
2],
[21.44052804, 29.70161639, 20.70460191, 39.90418045, 16.1483777
8],
[18.876308 , 26.38208059, 18.07969934, 35.23429953, 13.9782147
5],
[16.63960395, 23.47341883, 15.85530749, 31.15712074, 12.1048340
3],
[14.68819307, 20.92329068, 13.97555108, 27.59649208, 10.4886280
3],
[12.98525144, 18.68636627, 12.39173305, 24.48611006, 9.0952122
1],
[11.49893734, 16.72312788, 11.06186319, 21.7677198 , 7.8944503
8],
[10.2013182 , 14.9991189 , 9.94965275, 19.39216316, 6.8610655
7],
[ 9.06814642, 13.48429046, 9.02375611, 17.31505177, 5.9722389
1],
[ 8.0783105 , 12.15220147, 8.25686268, 15.49825178, 5.2084699
1],
[ 7.21342796, 10.98048099, 7.62613185, 13.90854714, 4.5528438
1],
[ 6.45749132, 9.94890861, 7.11117548, 12.51700172, 3.9906911
1],
[ 5.79649781, 9.04007002, 6.69464754, 11.29840502, 3.5092924
5],
[ 5.21843764, 8.23876281, 6.36161441, 10.23043133, 3.0974641
9],
[ 4.71266457, 7.53171188, 6.09923004, 9.29470762, 2.7459965
3],
[ 4.2699656 , 6.90732196, 5.89645437, 8.47412971, 2.4464179
8],
[ 3.88231419, 6.35546212, 5.74343167, 7.75415263, 2.1915576
6],
[ 3.54271656, 5.86727817, 5.63280675, 7.1220963 , 1.9752069
8],
[ 3.24507798, 5.43490383, 5.55722342, 6.56690368, 1.7919918
7],
[ 2.98408648, 5.05183908, 5.51072584, 6.07893135, 1.6372624
1],
[ 2.75511169, 4.71201421, 5.48822878, 5.64976711, 1.5069967
9],
[ 2.55407274, 4.41024533, 5.48539678, 5.27207134, 1.3977181
2],
[ 2.37754224, 4.14199391, 5.49854058, 4.93929274, 1.3064152
8],
[ 2.22240106, 3.90328437, 5.49854058, 4.64615961, 1.2305189
5],
```

7],	[2.08597116,	3.69063268,	5.56019917,	4.38761378,	1.1677679
1],	[1.9659162 ,	3.50098406,	5.56019917,	4.15939166,	1.1162304
1],	[1.86019735,	3.33165877,	5.65456113,	3.95776947,	1.0742408
7],	[1.76703503,	3.18030479,	5.65456113,	3.77949324,	1.0403644
3],	[1.6848755 ,	3.04481488,	5.76707568,	3.62171864,	1.0133664
8],	[1.61236193,	2.92346604,	5.76707568,	3.48195856,	0.9921845
8],	[1.54827557,	2.81461081,	5.88796313,	3.3580375 ,	0.9759063
6],	[1.49165092,	2.71684484,	5.88796313,	3.24805179,	0.9638371
3],	[1.44154595,	2.62893196,	6.01052473,	3.1503334 ,	0.9551305
6],	[1.39716889,	2.54978296,	6.01052473,	3.06343562,	0.9492955
8],	[1.35782745,	2.47843742,	6.1297553 ,	2.9860668 ,	0.9458421
9],	[1.32291594,	2.41404782,	6.1297553 ,	2.9171064 ,	0.9443508
1],	[1.29190414,	2.35586563,	6.24396137,	2.85557167,	0.9444642
1],	[1.26432775,	2.30322922,	6.24396137,	2.80060033,	0.9444642
1],	[1.23978001,	2.30322922,	6.3508372 ,	2.75143532,	0.9483363
1],	[1.21631899,	2.21232932,	6.3508372 ,	2.75143532,	0.9483363
6],	[1.18452807,	2.16000041,	6.42599795,	2.66794404,	0.9555366
6],	[1.18452807,	2.16000041,	6.42599795,	2.66794404,	0.9555366
1],	[1.13318689,	2.06863874,	6.48638983,	2.60076043,	0.9399102
1],	[1.11260153,	2.06863874,	6.48638983,	2.57212035,	0.9399102
4],	[1.09481028,	1.99654039,	6.54419164,	2.5463166 ,	0.9215784
4],	[1.09481028,	1.99654039,	6.54419164,	2.5463166 ,	0.9215784
3],	[1.06616841,	1.93941312,	6.60184769,	2.49127818,	0.9160054
3],	[1.06616841,	1.93941312,	6.60184769,	2.43906922,	0.9160054
9],	[1.04483472,	1.89394798,	6.720343 ,	2.43906922,	0.9147588
9],	[1.04483472,	1.89394798,	6.720343 ,	2.34902369,	0.9147588
7],	[1.02898749,	1.85759795,	6.83857359,	2.34902369,	0.9165012
7],	[1.02898749,	1.85759795,	6.83857359,	2.27459036,	0.9165012
9],	[1.01725269,	1.81985406,	6.95256857,	2.27459036,	0.9201555
9],	[1.01725269,	1.79042936,	6.95256857,	2.21200248,	0.9201555

3],	[0.9928709 , 1.76389064, 7.05940255, 2.21200248, 0.8942415
6],	[0.9928709 , 1.73994875, 7.05940255, 2.16012722, 0.8765515
9],	[0.94397815, 1.71832312, 7.1594923 , 2.16012722, 0.8608922
1],	[0.94397815, 1.71832312, 7.1594923 , 2.08437231, 0.8470975
3],	[0.9047733 , 1.68101729, 7.26817728, 2.08437231, 0.8349366
4],	[0.9047733 , 1.68101729, 7.26817728, 2.02342908, 0.8242078
2],	[0.87318415, 1.65040489, 7.37303851, 1.99725238, 0.8147348
8],	[0.87318415, 1.65040489, 7.37303851, 1.97362764, 0.8067000
7],	[0.84759141, 1.6251344 , 7.46883179, 1.95227467, 0.7992722
5],	[0.84759141, 1.6251344 , 7.46883179, 1.9329532 , 0.7926951
7],	[0.82675529, 1.60426962, 7.55384607, 1.91580885, 0.7868663
2],	[0.82675529, 1.59517034, 7.55384607, 1.89991049, 0.7816957
8],	[0.80971976, 1.58683314, 7.63009321, 1.88547395, 0.7771042
6],	[0.80242385, 1.57920843, 7.63009321, 1.87234908, 0.7730229
1],	[0.79680836, 1.57222923, 7.6978003 , 1.8604033 , 0.7697231
6],	[0.79262511, 1.56583488, 7.6978003 , 1.84951878, 0.7664684
2],	[0.78896224, 1.56009897, 7.75721391, 1.83959063, 0.7635611
2],	[0.78570632, 1.55470788, 7.75721391, 1.83052538, 0.7609648
1],	[0.78280811, 1.54975589, 7.81024413, 1.82260902, 0.7586435
3],	[0.78022535, 1.54520324, 7.81024413, 1.815007 , 0.7565656
1],	[0.77812765, 1.54101439, 7.8581529 , 1.80804229, 0.7551676
1],	[0.77662202, 1.5371574 , 7.8581529 , 1.80133817, 0.7539771
5],	[0.77518785, 1.53394764, 7.90183749, 1.80133817, 0.7530650
9],	[0.77392664, 1.5310427 , 7.90183749, 1.79040824, 0.7525137
4],	[0.77281869, 1.52840046, 7.94009152, 1.79040824, 0.7518016
1],	[0.77184349, 1.52597693, 7.94009152, 1.78244486, 0.7511677
2],	[0.77098442, 1.5237508 , 7.97306098, 1.78244486, 0.7506074
1],	[0.77022711, 1.52170425, 7.97306098, 1.7766095 , 0.7501111
7],	[0.76962644, 1.51982131, 8.00260195, 1.77418489, 0.7496709
6],	[0.76902882, 1.51808761, 8.00260195, 1.77143082, 0.7492802

```

7], [ 0.76850007, 1.51649019, 8.0284309 , 1.76946303, 0.7489330
2], [ 0.76818905, 1.51510615, 8.0284309 , 1.76746891, 0.7488446
1], [ 0.76814594, 1.51389806, 8.05028571, 1.7656052 , 0.7485724
3], [ 0.76815395, 1.51303897, 8.05028571, 1.76425854, 0.7483106
2], [ 0.76818596, 1.51227087, 8.06823422, 1.76243614, 0.7480770
1], [ 0.76823608, 1.51157391, 8.06823422, 1.76126836, 0.7478681
1], [ 0.76830148, 1.51093939, 8.0847021 , 1.76004467, 0.7476808
6], [ 0.76836425, 1.51040686, 8.0847021 , 1.75886454, 0.7475126
1], [ 0.7684446 , 1.50991356, 8.09799716, 1.75776824, 0.7473615
8]])

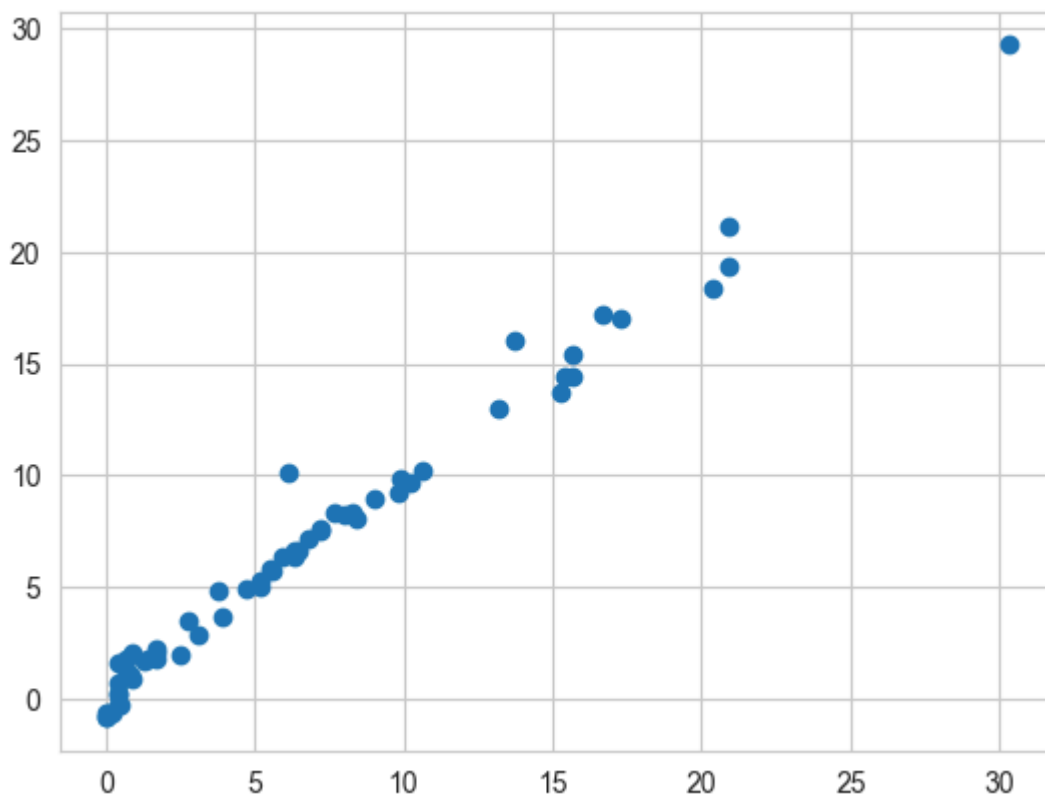
```

```

In [72]: y_pred=lasso_cv.predict(X_test_scaled)
plt.scatter(y_test,y_pred)
mae=mean_absolute_error(y_test,y_pred)
score=r2_score(y_test,y_pred)
print('MAE:',mae)
print('R2 score:',score)

```

MAE: 0.621195780721423
R2 score: 0.9817238521348797



```

In [73]: from sklearn.linear_model import RidgeCV
ridgecv=RidgeCV(cv=5)
ridgecv.fit(X_train_scaled,y_train)
y_pred=ridgecv.predict(X_test_scaled)
plt.scatter(y_test,y_pred)

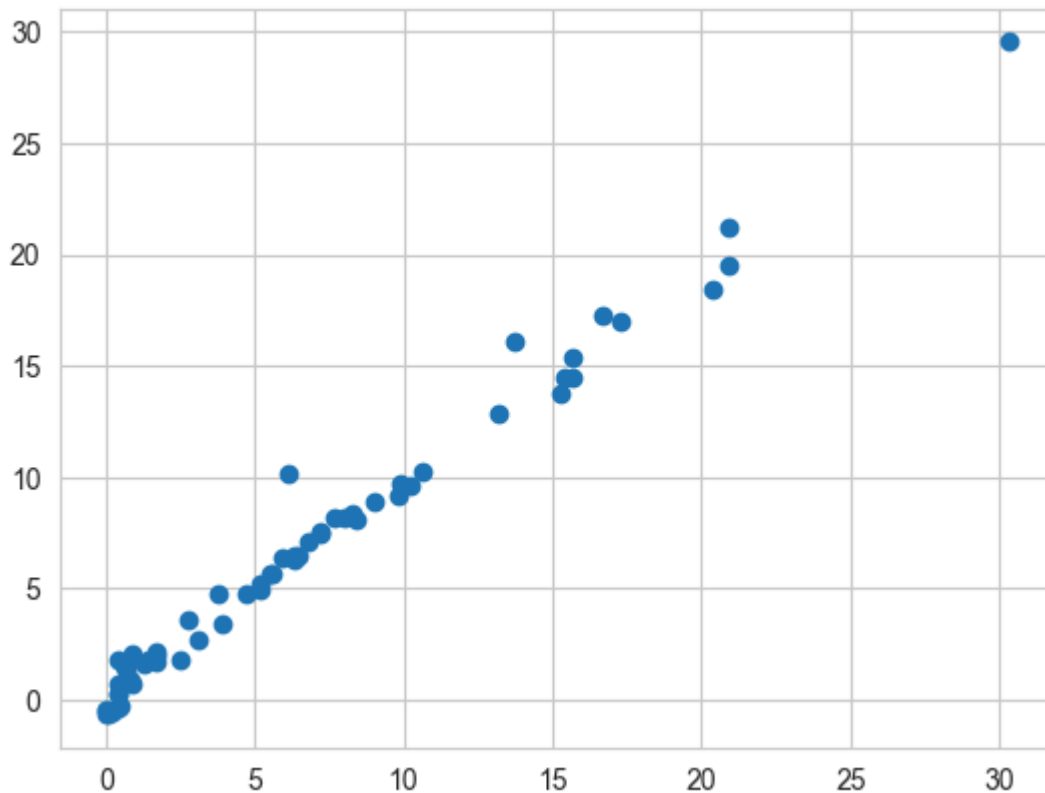
```

```

mae=mean_absolute_error(y_test,y_pred)
score=r2_score(y_test,y_pred)
print('MAE:',mae)
print('R2 score:',score)

```

MAE: 0.5969613029130073
R2 score: 0.9824907652592586



In [77]: `ridgecv.alpha_`

Out[77]: `np.float64(1.0)`

In [78]: `ridgecv.get_params()`

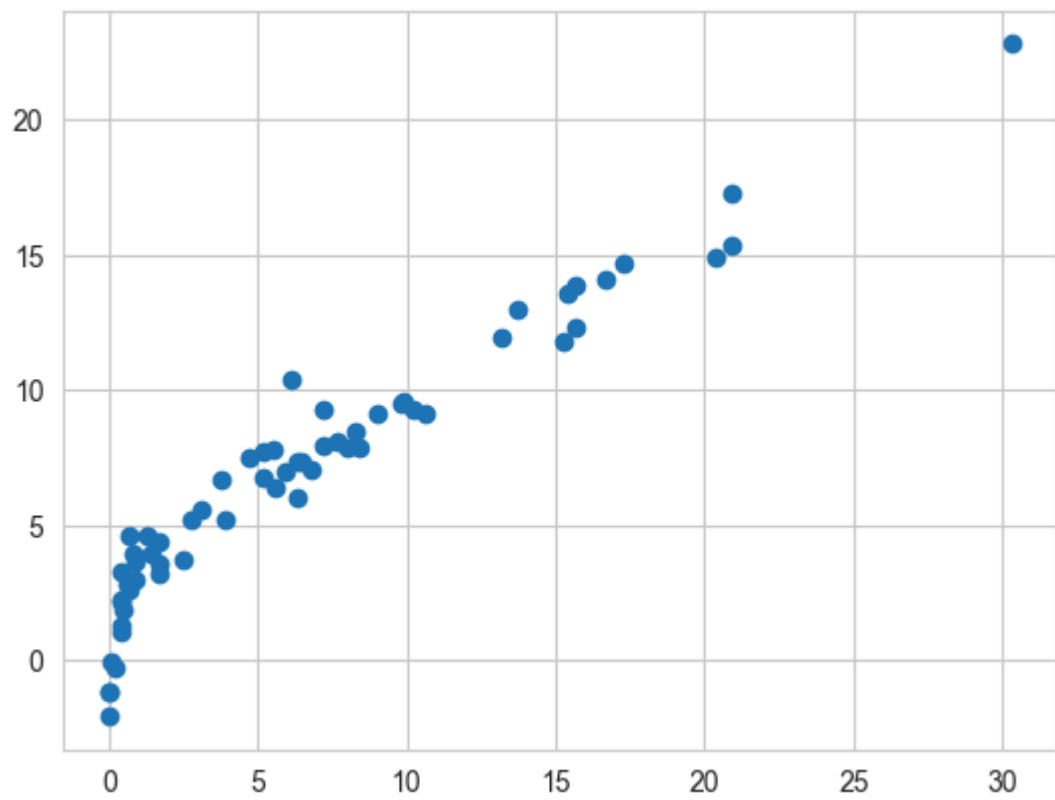
Out[78]: `{'alpha_per_target': False,
'alphas': (0.1, 1.0, 10.0),
'cv': 5,
'fit_intercept': True,
'gcv_mode': None,
'scoring': None,
'store_cv_results': False}`

```

In [80]: from sklearn.linear_model import ElasticNet
elasticnet=ElasticNet()
elasticnet.fit(X_train_scaled,y_train)
y_pred=elasticnet.predict(X_test_scaled)
plt.scatter(y_test,y_pred)
mae=mean_absolute_error(y_test,y_pred)
score=r2_score(y_test,y_pred)
print('MAE:',mae)
print('R2 score:',score)

```

MAE: 1.928441451179673
R2 score: 0.868664603256521



In []: